



Department of Computer Science
KISPECIxxx - Master Thesis

Thesis Title

Prepared by: Your name (xxxx@itu.dk)

Supervised by: Supervisor's name

Date: June 13, 2026

Abstract

This thesis proposes the design, implementation and evaluation of a survivable inspection UAV tailored for GPS-denied industrial environments. The project will develop a novel drone platform combining a lightweight, impact-tolerant protective structure that doubles as a contact sensor, an onboard perception stack for valve and inventory inspection, and a hybrid localization approach that fuses lidar/optical flow/IMU with UWB relays to enable robust close-proximity flight. Software work includes porting and integrating a PX4/ROS2 flight stack with Webots simulation to allow reproducible experiments, plus computer-vision pipelines for detecting valves, liquid levels and shelf items. The drone will be evaluated in simulation and physical tests on metrics including localization accuracy, inspection success rate, survivability (impact/drop resilience), and autonomy in docking/charging scenarios. The outcome aims to demonstrate that combining mechanical resilience with sensing-aware frame design and hybrid localization improves inspection reliability in confined, industrial settings – a practical step toward safer, more autonomous UAV inspection systems. The project aligns with the REAL LAB inspection topics and fulfils the MSc learning objectives by combining theory, tooling and empirical validation.

Will be now completely different, will be industrial settings, cluttered environments, survivability

lets see

None of this

Like this is the only thing that should work like that lol

mostly ok, except for the localization -> lets see how the localization by hitting things actually work

Contents

1	Introduction	IV
1.1	Research Questions	1
2	Related work	2
2.1	Collision-Resilient Drone Designs for Confined spaces	2
3	Methodology	4
3.1	Design and Development	4
3.1.1	Flight Type	4
3.1.2	Functional constraints and considerations	5
4	Physical design and considerations	12
5	Autonomous Drone Companion Computer Setup Workflow	13
5.1	Overview	13
5.2	Phase 1: OS and Network Setup	13
5.3	Phase 2: mDNS Hostname Resolution	14
5.4	Phase 3: Docker Installation	15
5.5	Phase 4: Python Virtual Environment	15
5.6	Phase 5: ROS2 Humble in Docker Containers	15
5.7	Phase 6: VSCode Remote Development	16
5.8	System Architecture	17
5.9	Key References	17
6	Next Steps: PX4 Integration	17
6.1	Hardware Setup: Pixhawk to Pi GPIO Connection	17
6.2	Install uXRCE-DDS Agent	19
6.3	Launch uXRCE-DDS Agent	19
6.4	Verify PX4-ROS2 Communication	19
6.5	Configure PX4 Parameters	20
6.6	Develop ROS2 Autonomous Control Nodes	20
6.7	Testing Methodology	20
6.8	SSH Authentication Setup for Containerized Development . .	21
6.8.1	SSH Key Generation (Host System)	21
6.8.2	Container Configuration (devcontainer.json)	21
6.8.3	SSH Client Installation (Dockerfile)	21
6.9	Troubleshooting Reference	22

6.9.1	Frame design	22
6.9.2	Cage design	22
6.10	Hardware	23
6.11	Software	23
7	Experiments and Results	23
7.1	Experimental Setup	23
7.1.1	Experiment Environment and Collision Target	23
7.1.2	Experiment Control and Data Collection	23
7.1.3	Pre-Computed Path and Waypoint Configuration	25
7.2	Experimental Kinematics and Path Deviation Analysis	25
7.3	Flight Stability and Efficiency Baseline	28
7.4	Velocity and Acceleration Dynamics	28
7.5	Actuator Saturation and Recovery Effort	31
7.5.1	Single-Flight Anatomy of Allocator Saturation	31
7.5.2	Aggregate Allocator Performance	31
7.6	The IMU Footprint of Impact Angles	33
7.6.1	Single-Flight IMU Impact Signature	33
7.6.2	Aggregate Angle-Dependent Signal	35
7.7	Environmental Sensing via Machine Learning	37
7.7.1	Data and Feature Engineering	37
7.7.2	Prediction Results	38
7.7.3	Implications for Contact-Based Environmental Sensing	41
8	Figures	42
9	Tables	42
10	Mathematics	42
10.1	Coordinate Frames	42
10.2	Drone Dynamics	43
10.3	Collision Kinematics	43

1 Introduction

The Broad Landscape of Aerial Inspection. The deployment of Unmanned Aerial Vehicles (UAVs) has revolutionized the field of industrial inspection. In expansive, outdoor environments, drones are now routinely utilized to inspect power lines, bridge infrastructure, and wind turbines, drastically reducing the risk to human operators while increasing efficiency. However, the transition of autonomous aerial robotics into indoor, confined spaces remains a significant challenge. Industries rely heavily on complex internal structures—such as large-scale storage vats, subterranean networks, and industrial piping—that require frequent inspection. Yet, the adoption of autonomous UAVs in these spaces severely lags behind outdoor applications due to a unique and punishing set of environmental constraints [28].

The Navigation and Computational Bottleneck. Operating a UAV within confined industrial spaces, such as a stainless steel storage tank, fundamentally alters the requirements for flight. These environments are routinely GPS-denied and, due to heavy ferromagnetism or physical shielding, completely signal-denied. This severs the link to human pilots, mandating full onboard autonomy. However, achieving autonomy in these conditions exposes the limitations of standard sensor suites. Featureless or highly reflective surfaces confound optical cameras, while traditional 3D LiDAR systems are often too heavy for the payload capacity of a drone small enough to enter the space [19]. Furthermore, even if the sensors could accurately map the environment, the computational weight required to run complex Simultaneous Localization and Mapping (SLAM) algorithms for real-time, 3D obstacle avoidance is immense—a challenge that even ground-based autonomous vehicles struggle to solve reliably in much simpler 2D environments [21].

From Avoidance to Exploitation: The Kinetic Penalty. Given the strict payload limits on onboard companion computers and the unreliability of traditional sensors in hostile environments, complex, real-time collision avoidance becomes computationally impractical. In tight spaces, collisions are not a symptom of system failure; they are an inevitability. Therefore, the paradigm of autonomous confined-space flight must shift from attempting to avoid obstacles at all costs to surviving them and, ultimately, exploiting them.

Surviving these inevitable impacts requires a mechanical design capable of significantly lowering the kinetic penalty of a collision. This could perhaps be achieved by a structural design that, to a certain extent, deflects the

collision forces away from the drone's main frame. While different solutions addressing this are already on the market [14], empirical data detailing their actual performance during flight are largely missing—a gap that must be investigated before physical impacts can be reliably exploited for navigation.

Summary and Warrant. If physical collisions in signal-denied environments are inevitable, and if a UAV can be designed to reliably survive them, a new avenue for spatial perception emerges. Rather than relying on heavy optical or laser-based sensors, a drone could theoretically utilize the very impacts it sustains to understand its surroundings. By interpreting the raw inertial data generated at the moment of collision, an autonomous system could extract foundational geometric information about its environment. This transition—from a drone that is merely blind and resilient to one that is blind but tactile—warrants a deep investigation into the mechanics of collision deflection and the interpretation of impact data.

1.1 Research Questions

To address the need for kinetic energy deflection and to investigate the potential for collision exploitation, this thesis details the physical and software development of a custom, autonomous quadcopter equipped with an onboard companion computer and a freely rotating protective cage. By mechanically decoupling the protective outer shell from the drone's main frame, this design introduces a mechanism to potentially deflect rotational energy and mitigate impact penalties.

Using a highly controlled, ground-truth motion capture (MOCAP) environment to validate the system, this research seeks to answer the following questions:

1. How does the physical resilience and energy deflection capability of a freely rotating protective cage compare to a fixed cage during direct obstacle collisions?
2. To what extent can the raw inertial data generated during these deliberate physical collisions be utilized to accurately estimate the angle of impact against an obstacle, serving as a fundamental step toward tactile spatial awareness in signal-denied environments?

2 Related work

2.1 Collision-Resilient Drone Designs for Confined spaces

Here mention the congifly [15], spinfly[20] as something I mainly continued and learned from , Flyability[9] cluttered environmnet inspiration to how to build the cage [22] talk about gomboc and what not FOR NEXT TIME -> start easy on the first 2 things, perhaps talk about th edevelopment of the drone first in the Methodology section

This paper is a spiritual continuation of Spinfly [20] by A. Lerche. Lerche had a a similiar problem statement and he tried to develop a drone for autonomous inspection, in particular odometry. From his work and personal conversations with him a number of lessons to avoid in this work has been identified. Lerche used iNAV flight control framework as backbone of his drone development. While iNAV allows for autonomous flight it si not particularly well suited for fast development. Lerche had spent a considerable time and resources on hardware connections, particularly soldering of Flight Controller and other devices together and various troubleshooting of hardware and software in iNAV. Constraint of navigating cluttered spaces safely brought me to work on Robust Aerial Swarms by [25] where instead of active obstacles detection and avoidance they instead crashed drones and have them recover from the crash, albeit with drone swarms each weighing 25grams. This takes a radically different approach to solving 'how to get from A to B' as Simultaneous localization and visualization (SLAM) is at a forefront of robotics and in many ways its holygrail, Mulgaonkar has instead chosen to map environment by clashing with it, and instead vicariously map it by localization of the drones themselves. And instead of relaying on detecting obstacle first and never clashing with it, have resilient enough drone which can instead recover from the clash and continue its mission. As SLAM is being developed it is not a ready technology for relatively small drones. Even best automakers in the world have still not brought this technology to navigate reliably any environment at the moment with state of the art LIDARs and other expensive and somewhat heavy equipment. While unmapped and changing environment of potential drone use in cluttered environment can be many fold more difficult to navigate and thus require quite sizable, heavy and expensive equipment if the drone was to go down the SLAM road for autonomy. While small drones in the aerial swarms could navigate their environment with clashing, using a a simple cage, the drone proposed in this

work should be able of partially autonomous flying when being denied signal and even have ability to inspect its environment with various sensors it might use for it. This warrants some functional payload overhead which due to lift physics can be overcome only so much by more powerful motors, and at certain point the drone must grow in size. For this project a 4inch propeller drone design was chosen based on criteria mentioned later. At this size the penalty of hitting something increases substantially *DATA???? citetions? soemthing to prove this claim???* as the ratio of inertia and relative structural strenght of the drone and its propellers decreases, and on the other hand with the increased size of the drone everything in its environment is relatively smaller. Meaning that while Mulgaonkar could rely on simple 2 tube cage spanning their drones to avoid most items in their environment, the protection of a 4inch drone (4inch propellers -> at minimum 20cm in X and Y dimmensions) must be such that this drone is protected from corners of items such as shelves or continuous pipping and that such objects do not directly interfere with the drone, and with its propellers in particular. Here a research 22, Development of Collision Resilient Drone for Flying in Cluttered Environment helps to tackle this issue. He proposes either a gimbal like sphere which freely spins upon contact with its environment, similar as a commercial product by Flaybility [9] *Sourcing missing, fucked up ZOTERO* and a Gombok [3] *eve fucking wikipedia reference will do here*. Gombok shape, or in this case a turtle-like shape has an interesting property of being potentially stable only in one orientation, when a turtle is turned upside down it can just by its own movement come back to its feet thanks to its center of gravity being suddenly high up in such case, making it more unstable. Mahmout has leveraged these properties in their design as well and concluded his paper with it being the superior cage design of the two. A specific use case given by supervisor of this paper comes to mind: inspection of stainless steel tanks. Today this is executed by a human who needs to descent into the tank through a menhole on the top. Hypothetically a turtle shaped drone cage, which would after inspecting the tank ascend to fly out through the menhole, could more easily get through the narrow opening even if it would not navigate perfectly out of the hole, as if only the center of the drone would be within the manhole limits the drone could somewhat stably ascend without it being rocked too hard, as it would leverage its shape and its low center of gravity. Due to this hypothesis, the resulsts Mahmout had with their cage design, relative simplicity opposed to gimbal sphere and even possibility of using it similarly as Lerche did for odometry, the spinning turtle

shape was eventually chosen for the drone design.

Apart from physical survivability of larger drones in cluttered environments a topic of software control for autonomous flight had to be addressed. While this is a frontier technological development topic, 2 autonomous flight platforms stand out with their rate of adoption and technological maturity, ArduPilot and PX4. Both are widely adopted and have a lively base of developers who add to their codebases. Apart from more free licensing of PX4 (which even allows for commercial extension based on the PX4 for proprietary purposes, while Ardupilot requires all development to be added to their codebase CITATION MISSING AGAIN) PX4 differs from Ardupilot having more modularized architecture and having more well developed hardware standards. This gave PX4 the edge in this work as it allowed for more straightforward attitude to the development and using standardized parts presumably with no or little troubleshooting.

3 Methodology

The methodology for RATFLY project stands on development of drone to use for the various tests and implementations together with controlled environment to test the drone in. As to the development, it consists of hardware choices, software choices and the constraints sprouting from these two which dictate much of the development.

3.1 Design and Development

3.1.1 Flight Type

A quadcopter design was a very clear choice as these are widely used and allow for very nimble and precise movement as opposed to fixed-wing designs which need to be in forward motion to generate upthrust for their flight. The Quadcopter design due to its symmetric nature make it very easy to predictably control it.

On figure Figure 1 you can see standard quadcopter setup. If front motors (m_1, m_2) spin comparatively more the drone pitches up, if side propellers (m_1, m_4) spin comparatively faster, then the drone rolls towards its left side, and thanks to the setup of alternating spin direction if propellers diagonally from each other m_1 and m_3 will spin faster, while m_2 and m_4 will spin slower the drone will turn on around in the direction of the propellers which

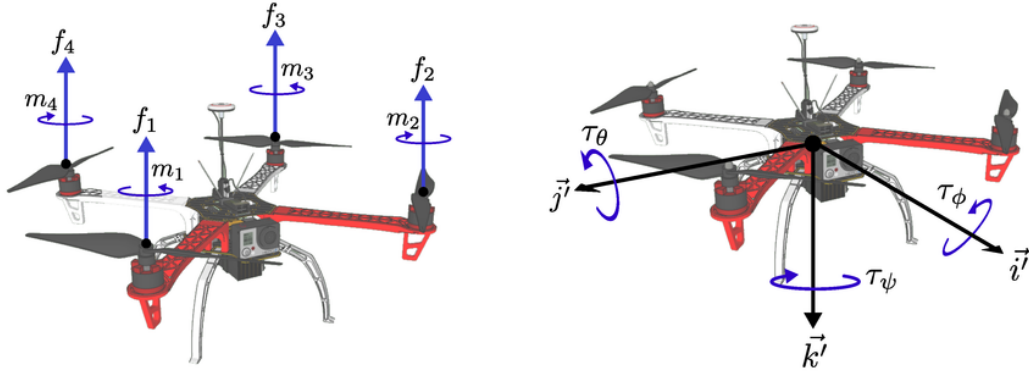


Figure 1: typical quadcopter setup 'props-in', yaw ψ (turning around), pitch θ (nose up or down) and roll ϕ (tilting to sides) From [18]

are faster, in this case turn towards its left. This is given by Newton's first law as the inertia of propellers spinning faster will induce the rotation in the propellers' direction. Opposingly, pitch and roll is achieved by spinning pairs of propellers on one side faster than on the other (left vs right, front vs back). This makes the control of quadcopter somewhat simple as it can be summarized in a mathematical model.

3.1.2 Functional constraints and considerations

As this work is in certain sense a continuation of Lars [20] Spinfly paper, lessons learned from his project were well considered. One of the largest limitations mentioned in the work and also by the author himself in person was insufficient weight headroom of the drone which had trouble taking off and maintaining the flight when fitted with the cage. Another one was insufficient battery life which did not allow for prolonged flight and made subsequent tests difficult to run. Both of these problems are solved in tandem as battery capacity and its weight are directly proportional and can have diminishing returns if the battery has high capacity but get drain faster as the motors have to output more power. Lerche also reflected on lack of robust autonomous controller on board and troubles with telemetry realying the communication. [I want to make this more structured, maybe better extract the infomarion, maybe summap that was I needed to do was

1. Carry bigger load

2. Fly longer/more efficiently
3. Simpler and more robust flight control [FC]

Size of 6C FC is substantial enough to warrant a 4inch drone design, while typically drones of this size, 4inches is also the upper maximum size to be flown indoors [2]. The ratio of thrust-to-weight for common drones is between 4:1 to 3:1, meaning maximum thrust of the the motors combined should be 3-4times greater than the weight of the drone. Due to needed weight headroom, which by estimation and after discussion with Spinfly author, I set to be 800-1000g. I chose Emax ECOII 2004 1600KV motors. These are typically recommended for 4.5inch to 5inch ultalight drone builds, but similiar motor by a different company has been tested with 4inch propellers GF4023 propellers and proven to 860g of thrust per motor, combined 3440g per drone giving me 1:3.44-1:4.30 ratio depending on the total weight. This also means the whole drone can stay hovering with approximately 50% of thrust which is recommended due to the motors being twice as efficient than at 100% thrust. Designation 2004 means it has 20mm diameter and 4mm in height, this is significant diameter in the class which makes it slower to respond and adjust RPMs, on the other hand it allows for higher thrust at 50% throttle. 1600KV metric is how many RPMs are produced when applying 1V. 1600KV is at the low end of KV spectrum and was selected in accordance with using a 6 cell battery which provides 24V providing ca 38400RPM ($1600 * 24$) which falls well within recommended speeds for 4inch propellers. 6S battery is also an important choice from the efficiency perspective as running higher voltages draws less current and the whole setup keep a lower temperature as opposed to 4S setups. Additionally there is voltage sag phenomenon (LLM here-> explain it for me pls) that is less of a factor with higher voltages setups To address this holistically with a focus on the my research question I choose PX4 standard for my autonomous flying solution. PX4 is a an open source standard with modular architecture uORB (Micro Object Request Broker) which is suited for plug-and-play development as it's high modularity allows for faster research, as the physical Flight Controller (FC) manufactured under this standard Pixhawk 6C, as opposed to most FCs, so the accompanying electronics can be quickly swapped, tried and iterated on. This Pixhawk 6C FC has been chosen as a solid FC solution with IMU and compass sensors already integrated, simplifying the drone development and allowing me to concentrate on the research question itself. [I feel like this should be]

Iterative Learning from Predecessor Work

The design began with critical constraints identified in the Spinfly platform [20]. Two primary limitations directly shaped this work:

1. **Insufficient weight headroom** – The original platform struggled with takeoff and stable flight when equipped with the protection cage, leaving no margin for component upgrades or mission adaptations.
2. **Insufficient battery capacity** – Limited flight time created operational friction; test iterations became difficult to schedule, reducing research throughput.

These problems are **not independent**—they form a coupled system. Battery capacity and its mass are directly proportional, yet simply increasing capacity can create diminishing returns. A heavier battery forces motors to output more power, which accelerates discharge rate, potentially wasting the added capacity. The design needed to solve both simultaneously through systematic trade-off analysis.

Lerche also highlighted secondary constraints: lack of robust autonomous flight control on the flight controller itself, and telemetry transmission reliability. These informed the autopilot platform selection.

Design Objectives: Based on these constraints, the design targets were clarified into three interconnected goals:

1. **Carry larger payloads** – The base payload must be much larger, to allow for cage development and to leave weight headroom for any additional development
2. **Achieve longer flight duration** – Extend mission time while maintaining efficient power consumption.
3. **Simplify autonomous control** – Use a proven, modular flight controller stack to reduce software complexity and improve reliability.

Platform Selection: PX4 and Pixhawk 6C To address these objectives holistically, PX4 was selected as the autonomous platform. PX4 is an open-source standard built on modular architecture using **uORB (microObject Request Broker)**—a publish-subscribe middleware layer suited for plug-and-play development.[24] This modularity enables rapid research iteration:

novel autonomous behaviors can be implemented as independent modules without modifying the core flight stack.

The **Pixhawk 6C flight controller** was chosen as the hardware instantiation of the PX4 standard. Critically, Pixhawk is not a monolithic design but represents standardized connector specifications, sensor integration points, and reference schematics managed by the Dronecode Foundation. This standardization means:

- **Hardware swappability:** Sensors and peripheral electronics can be upgraded or replaced with components from different manufacturers (Holybro, ARK Electronics, CUAV) without redesigning integration.
- **Ecosystem maturity:** Over 1 million Pixhawk-based devices are deployed across commercial, research, and hobbyist applications.[31]
- **Sensor integration:** The Pixhawk 6C includes integrated IMU and compass. Specifically, it features redundant inertial measurement units with onboard ICM-42688-P and BMI088 accelerometers/gyroscopes, plus an IST8310 magnetometer, with temperature-controlled operation via onboard heating resistors.[5–7] This eliminates the need for custom sensor fusion code—allowing focus on the research question rather than flight control tuning.

The 6C’s form factor ($84.8 \times 44 \times 12.4$ mm) justifies the 4-inch drone design choice.[7] Four-inch platforms represent the upper practical limit for indoor flight while remaining large enough to carry meaningful payloads.

Thrust-to-Weight Ratio: Balancing Performance and Efficiency Definition and Target Range

Thrust-to-weight (TWR) ratio expresses the total available thrust divided by drone mass. It directly determines hover efficiency and responsiveness. Industry guidelines establish a spectrum across applications.[26]

Application	TWR	Hover Throttle	Notes
Aerial photography	2:1 to 3:1	40–50%	Stable, efficient
Research/cargo	3:1 to 4:1	30–50%	Responsive, good efficiency
Racing/freestyle	5:1+	15–25%	Aggressive maneuvers

Why TWR Matters: Efficiency and Control Authority

Motor efficiency does not follow a simple linear relationship with throttle percentage. Instead, efficiency curves show a **peak efficiency at mid-range throttle** (typically 40–70%), with degradation at both very low and very high throttle levels.[8] This is critical for autonomous drones.

If a drone hovers at 100% throttle, it has zero authority to climb, stabilize, or reject wind gusts—any disturbance forces maximum output, leaving no control margin and risking loss of stability during maneuvers.[16] Conversely, hovering at 40–50% throttle reserves substantial available thrust for autonomous obstacle avoidance, gust rejection, and rapid maneuvers required during precision autonomous flight.[30]

Motor Selection: EMAX ECOII 2004 1600KV Why This Motor?

The EMAX ECOII 2004 series is specified for 4–5 inch ultralight builds.[1] The 2004 designation specifies stator dimensions: 20mm diameter and 4mm height.[1] This dimensional choice has performance implications explained below.

Pilot testing of a similar 2004-series motor (T-Motor F2004 1700KV, comparable in stator dimensions) paired with 4-inch GF4023 propellers and a 6S battery demonstrated approximately 860 grams of static thrust per motor.[17, 26] With four motors, this yields combined thrust capacity of approximately 3,440 grams—establishing the system’s TWR.

TWR Calculation (estimated final weight 800–1,000g):

- Total available thrust: 3,440g
- Target weight: 900g (midpoint)
- $\text{TWR} = 3,440 / 900 = \mathbf{3.82:1}$

This places your design in the research/autonomous platform zone. A TWR of 3.8:1 means the drone can sustain controlled hovering well within the efficient mid-throttle operating range.

Understanding Motor Nomenclature

The **2004 designation** specifies 20mm stator diameter and 4mm height. Larger stator diameters generate more torque but have slower transient response (RPM changes take longer to complete) compared to smaller-diameter motors.[23] This trade-off is deliberate: at mid-throttle operation, the 2004’s larger stator sustains higher efficiency than smaller, higher-KV alternatives

that must spin faster to produce equivalent thrust. For autonomous flight requiring smooth, predictable thrust response, this characteristic is desirable.

The **1600 KV rating** indicates 1600 RPM per applied volt. This low-KV designation was selected specifically for your 6S battery voltage.

Battery Selection: 6S LiPo Why 6S and Not 4S?

Battery voltage selection cascades through the entire drivetrain. The **6S (six-cell) LiPo pack** produces nominal voltage of 22.2V (6 cells $\times$ 3.7V per cell).[11] This voltage selection determines optimal motor KV and operating efficiency.

1600 KV with 6S Battery:

- Theoretical no-load RPM: $1600 \text{ KV} \times 22.2\text{V} = \mathbf{35,520 \text{ RPM}}$ [8]
- Practical RPM under propeller load: $\sim 24,000\text{--}28,000 \text{ RPM}$ (typically 70–80% of theoretical no-load speed due to air resistance and propeller drag)[26]
- This range is optimal for 4-inch propellers, avoiding excessive tip speeds that reduce aerodynamic efficiency

Higher voltage = less current for equivalent thrust. When motors operate on 4S ($\sim 14.8\text{V}$ nominal), they require significantly higher current draw to achieve the same thrust.[27] Higher current increases internal losses in battery, ESCs, and motor windings—manifesting as excess heat and reduced flight time. 6S draws proportionally less current for equivalent thrust, keeping system temperature lower. This is measurable: one study comparing 4S vs 6S motor systems on the same mechanical load found that 6S exhibits “greater low-end control” with lower current heat generation under identical thrust demands.[29]

Voltage Sag: The Critical Factor for Autonomous Control What is Voltage Sag and Why It Matters

Voltage sag refers to the **temporary voltage drop that occurs during high-current discharge events**. This drop is caused by battery internal resistance (IR). Ohm’s Law describes this relationship: **Voltage Drop = Current $\times$ Internal Resistance**.[10]

Every LiPo battery contains internal resistance—even high-quality cells. When you demand high current from the battery (e.g., during rapid acceleration), that current flowing through internal resistance creates a voltage drop.[27] For example:

- A 4S battery (14.8V nominal) with typical internal resistance might sag from 14.8V down to **10.2V** during peak acceleration
- A 6S battery (22.2V nominal) with equivalent quality, under the same peak current draw, experiences proportionally less sag (from 22.2V down to ~16.5V)[4]

The absolute sag magnitude (difference in volts) may be similar between 4S and 6S. **But the percentage impact is vastly different:**

- 4S sag example: 14.8V $\rightarrow$ 10.2V is a **31% voltage loss**
- 6S sag example: 22.2V $\rightarrow$ 16.5V is only a **26% voltage loss**

Why This Matters for Autonomous Flight Controllers

Your Pixhawk 6C relies on stable voltage to maintain consistent motor response during rapid maneuvering. The flight controller sends throttle commands expecting motors to accelerate at a predictable rate. **Excessive voltage sag introduces latency between commanded acceleration and actual motor output.** Here's the causal chain:

1. Flight controller commands 70% throttle for obstacle avoidance maneuver
2. Motor voltage sags unexpectedly from 22V to 16V due to high current draw
3. Motor RPM drops below what the controller expected
4. Actual thrust output is lower than the controller's state estimator predicted
5. The control loop detects the mismatch and must recalculate—introducing lag

During high-speed autonomous maneuvers, this lag degrades control stability and can risk uncommanded behavior. Higher baseline voltage (6S) provides a larger voltage margin, reducing the percentage impact of sag on motor performance.[27]

6S Enables Efficient Hover at Higher Throttle Margin

Since 6S provides higher baseline voltage, your motor's hover throttle point is positioned differently on the efficiency curve compared to 4S. Hover throttle percentage depends on specific motor and load characteristics—**there is no direct proportionality between voltage and hover throttle**. However, empirical testing shows: **“6S is a clear winner in efficiency up to around 45 throttle [percentage].”**[29]

Your 1600KV motor on 6S should hover in the **45–50% throttle range** based on your estimated drone weight and motor thrust.[23] This operating point—hovering near the motor efficiency peak—means you extract maximum flight time from your battery while maintaining responsive control authority (50% throttle reserves 50% available thrust for maneuvers and environmental disturbances).

System Integration: How Components Work Together This configuration is tightly coupled—changing any single parameter cascades inefficiencies throughout the system:

If you switched to 4S battery, for example, you would need lower-KV motors (e.g., 1100KV) to maintain similar RPM. But lower-KV motors produce less torque, potentially requiring larger propellers or higher current draw—both of which degrade efficiency. Every selection interconnects. Your component choices represent a validated system point.

4 Physical design and considerations

The

Parameter	Your Design	Rationale
Frame size	4-inch wheelbase	Pixhawk 6C form factor ($84.8 \times 44 \times 12.4$ mm) + indoor flight compliance
Motor KV	1600 KV	Produces 24,000–28,000 RPM under load on 6S; optimal for 4-inch propellers[26]
Battery voltage	6S (22.2V nominal)	Reduces voltage sag percentage impact; improves efficiency at cruise throttle[29]
Hover throttle	~45–50%	Operates at motor efficiency peak; reserves 50% available thrust[23, 29]
Estimated TWR	3.82:1	Responsive yet efficient for research autonomous missions

5 Autonomous Drone Companion Computer Setup Workflow

5.1 Overview

Setup a Raspberry Pi 5 running Ubuntu 24.04 with ROS2 Humble in Docker containers for autonomous drone development with PX4 flight controller integration. The containerized approach solves Ubuntu 22.04/Pi 5 kernel incompatibility while providing ROS2 Humble support.

5.2 Phase 1: OS and Network Setup

Flash Ubuntu 24.04 Desktop to microSD using Raspberry Pi Imager with the following configuration:

- Hostname: `pi5drone` (appears as `dorten-pi5drone` in system)
- Username and password configured

- SSH enabled via Services tab (critical step)
- Wi-Fi configured for local network
- Locale: Europe/Copenhagen, Country: DK

Note: Raspberry Pi 5 uses micro-HDMI ports for display output, not USB-C. USB-C is power-only (27W PSU recommended).

Boot the Pi and verify connectivity. SSH server is not auto-started despite Imager setting—manual installation and enablement is required:

```
sudo apt install openssh-server
sudo systemctl enable --now ssh
```

5.3 Phase 2: mDNS Hostname Resolution

Problem: Connecting via `ssh dorten@dorten-pi5drone.local` failed with name resolution errors.

Solution: Install Avahi daemon on both Pi and workstation for multicast DNS (mDNS) support.

On Raspberry Pi:

```
sudo apt install avahi-daemon
sudo systemctl enable --now avahi-daemon
```

On workstation (Ubuntu):

```
sudo apt install avahi-daemon libnss-mdns
sudo systemctl enable --now avahi-daemon
```

Verify hostname resolution:

```
ping dorten-pi5drone.local
```

Optional enhancement—Passwordless SSH: Execute `ssh-copy-id dorten@dorten-pi5drone.local` from workstation (one-time operation eliminates password prompts for subsequent connections).

5.4 Phase 3: Docker Installation

Install Docker Engine following official documentation [12]:

```
sudo apt install docker.io docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Known issue: Ubuntu 24.04 ships with containerd package that conflicts with Docker's containerd.io. Remove before installation:

```
sudo apt remove containerd
```

Configure rootless Docker to allow non-root user container execution [13]:

```
sudo groupadd docker
sudo usermod -aG docker $USER
newgrp docker
```

Verify installation:

```
docker run hello-world
```

5.5 Phase 4: Python Virtual Environment

Ubuntu 24.04 implements PEP 668 externally-managed-environment policy, blocking system-wide pip installations. Create isolated Python environment:

```
sudo apt install python3.12-venv
python3 -m venv ~/ros2_env
source ~/ros2_env/bin/activate
pip3 install vcstool
```

Note: Virtual environment activation required for each session, or can be configured in shell startup files.

5.6 Phase 5: ROS2 Humble in Docker Containers

Rationale: Ubuntu 22.04 is the official platform for ROS2 Humble binary packages. Direct installation on Raspberry Pi 5 running Ubuntu 24.04 is not officially supported due to kernel incompatibility with RP3A0-M1 hardware. Containerization provides Ubuntu 22.04 environment running inside the 24.04 host, enabling native ROS2 Humble support without source compilation.

Create workspace structure:

```
mkdir -p ~/ws/.devcontainer
cd ~/ws
```

Create `Dockerfile` providing base image with Ubuntu 22.04 + ROS2 Humble (using official `ros:humble` image from Open Source Robotics Foundation), non-root user account with UID matching host system, passwordless sudo configuration, and development tool installation.

Create `.devcontainer/devcontainer.json` defining Visual Studio Code container configuration. Key settings:

- `-net=host, -pid=host, -ipc=host`: Required for ROS2 DDS discovery
- `ms-iot.vscode-ros`: ROS2 syntax highlighting and tooling
- `privileged: true`: Enables GPIO and UART access for PX4 flight controller communication
- X11 forwarding mounts: Support for graphical applications (RViz2, rqt)
- `postCreateCommand`: Executes rosdep dependency resolution, workspace permission fixes, and ROS2 environment setup

5.7 Phase 6: VSCode Remote Development

On development workstation:

1. Install “Remote - SSH” and “Dev Containers” extensions in Visual Studio Code
2. Command Palette (Ctrl+Shift+P) → “Remote-SSH: Connect to Host” → `dorten@dorten-pi5drone.local`
3. VSCode opens remote window connected to Pi
4. Open folder `/home/ws`
5. VSCode detects `.devcontainer/devcontainer.json`, prompts “Re-open in Container”
6. VSCode builds Docker image (first execution: approximately 30–50 minutes on Raspberry Pi 5)

7. Container starts and initializes; development environment ready

Verification of containerized environment:

```
lsb_release -a          # Expected: Ubuntu 22.04 LTS
ros2 --version          # Expected: ROS 2 Humble
echo $ROS_DISTRO        # Expected: humble
```

5.8 System Architecture

The complete development architecture is illustrated in Figure 2.

5.9 Key References

- Docker installation: <https://docs.docker.com/engine/install/ubuntu/>
- Rootless Docker configuration: <https://docs.docker.com/engine/install/linux-postinstall/>
- ROS2 Docker VSCode guide: <https://docs.ros.org/en/humble/How-To-Guides/Setup-ROS-2-with-VSCode-and-Docker-Container.html>
- VSCode Remote SSH: <https://code.visualstudio.com/docs/remote/ssh>
- PX4 companion computer: https://docs.px4.io/main/en/companion_computer/pixhawk_rpi
- uXRCE-DDS middleware: https://docs.px4.io/main/en/middleware/uxrce_dds

6 Next Steps: PX4 Integration

6.1 Hardware Setup: Pixhawk to Pi GPIO Connection

Connect Pixhawk TELEM2 port to Raspberry Pi GPIO pins for UART serial communication (921600 baud):

Verify UART is enabled on Raspberry Pi and that device node `/dev/ttyAMA0` is accessible to the container user.

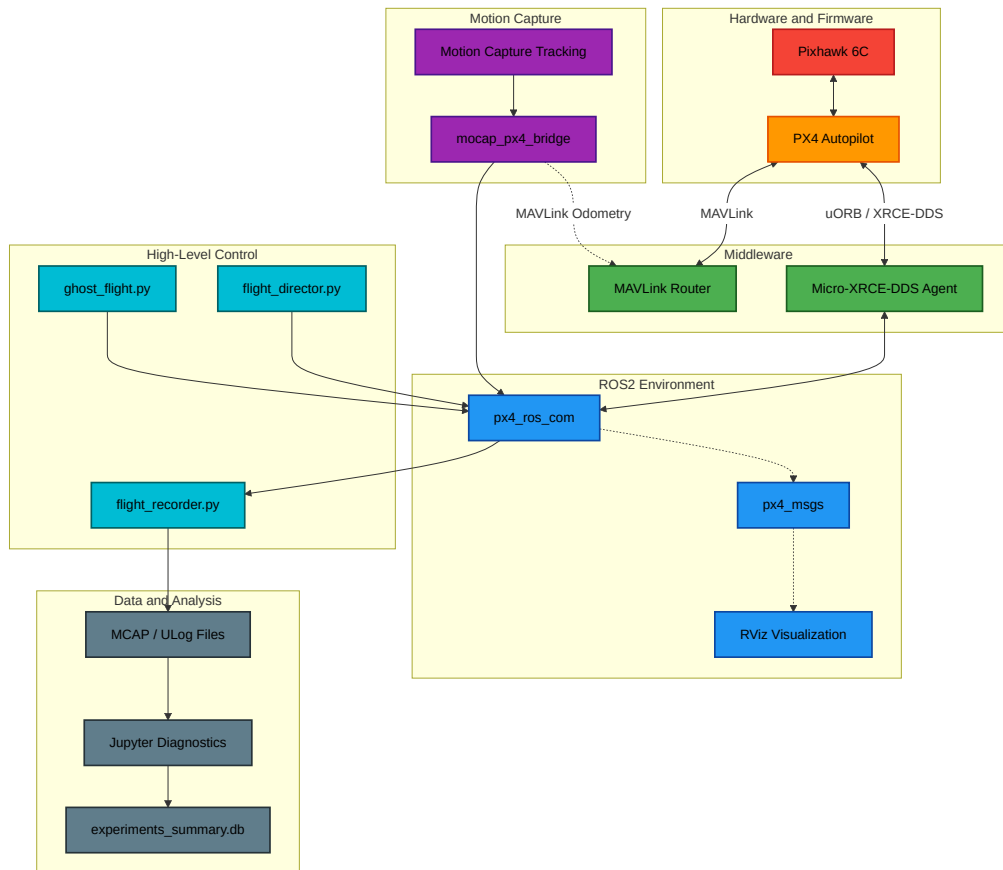


Figure 2: Drone software stack architecture with PX4 flight controller, ROS2 middleware, MOCAP integration, and data analysis pipeline

Pixhawk TELEM2	Pi GPIO Pin	Function
TX (pin 2)	GPIO 15 / RXD (pin 10)	Receive data
RX (pin 3)	GPIO 14 / TXD (pin 8)	Send data
GND (pin 6)	Ground (pin 6)	Common ground

Table 1: Pixhawk TELEM2 to Raspberry Pi GPIO wiring for uXRCE-DDS communication

6.2 Install uXRCE-DDS Agent

Inside VSCode terminal (already within container):

```
sudo apt install ros-humble-micro-xrce-dds-agent
```

The uXRCE-DDS Agent acts as a bridge between the PX4 flight controller (running uXRCE-DDS client) and the ROS2 middleware, translating between uORB messages and ROS2 topics.

6.3 Launch uXRCE-DDS Agent

Start serial connection to Pixhawk:

```
MicroXRCEAgent serial --dev /dev/ttyAMA0 -b 921600
```

Expected output: Agent status indicating successful session creation once Pixhawk boots and establishes connection.

6.4 Verify PX4-ROS2 Communication

In new VSCode terminal (container), list active ROS2 topics:

```
ros2 topic list
ros2 topic echo /fmu/out/vehicle_odometry
```

Expected: Topics from PX4 flight controller such as /fmu/out/vehicle_odometry, /fmu/out/vehicle_status, /fmu/out/vehicle_attitude, etc.

6.5 Configure PX4 Parameters

Using QGroundControl on development workstation, configure PX4 firmware parameters to enable uXRCE-DDS middleware:

- **UXRCE-DDS_CFG:** Set to TELEM2 (serial port configuration)
- **SYS_AUTOSTART:** Select appropriate vehicle type (quadcopter, fixed-wing, etc.)
- Verify serial connection parameters match uXRCE-DDS Agent configuration (921600 baud)

6.6 Develop ROS2 Autonomous Control Nodes

Implement mission planning and autonomous flight logic in ROS2:

- Subscribe to PX4 telemetry topics: `/fmu/out/vehicle_odometry`, `/fmu/out/vehicle_status`
- Publish control commands to PX4: `/fmu/in/offboard_control_mode`, `/fmu/in/trajectory_setpoint`
- Implement waypoint navigation, obstacle avoidance, and mission state machines in Python or C++
- Integrate sensor inputs (GPS, IMU, cameras) from PX4 uORB message stream via ROS2 topics

6.7 Testing Methodology

1. **Simulation First:** Use PX4 Software-in-the-Loop (SITL) simulator to test autonomous algorithms without real hardware risk
2. **Monitor Telemetry:** Subscribe to PX4 topics in ROS2 to verify correct message flow and telemetry accuracy
3. **Hardware Validation:** Deploy ROS2 nodes to Raspberry Pi container once simulation testing successful
4. **Flight Testing:** Monitor autonomous flights via MAVProxy or QGroundControl; collect flight logs for analysis

6.8 SSH Authentication Setup for Containerized Development

Purpose: Enable Git operations from within a Docker container using host SSH credentials.

Challenge: Docker containers are isolated environments. By default, a container cannot access the host system's SSH keys needed for GitHub authentication.

Solution implemented in three steps:

6.8.1 SSH Key Generation (Host System)

```
ssh-keygen -t ed25519 -C "your-email@example.com"
```

Generated an ED25519 key pair on the Raspberry Pi 5 host system. The public key (`id_ed25519.pub`) was added to GitHub account settings under SSH keys, enabling passwordless authentication.

6.8.2 Container Configuration (`devcontainer.json`)

```
"mounts": [  
  "source=${localEnv:HOME}/.ssh,target=/home/${localEnv:USER}/.ssh,  
  type=bind,readonly"  
]
```

Mounted the host's `.ssh` directory into the container as read-only. This allows the container to access SSH keys without duplicating or exposing them.

6.8.3 SSH Client Installation (`Dockerfile`)

```
RUN apt-get update && apt-get install -y openssh-client && \  
  rm -rf /var/lib/apt/lists/*
```

Added OpenSSH client to the container image, enabling Git to use SSH protocol for remote operations.

Result: The development container can authenticate with GitHub using the host's SSH keys, enabling seamless `git push`, `git pull`, and `git clone` operations without credential prompts.

Why SSH over HTTPS: SSH keys provide secure, passwordless authentication and persist across sessions, eliminating the need to store tokens or enter credentials repeatedly during development.

6.9 Troubleshooting Reference

Issue	Likely Cause	Resolution
SSH connection fails	SSH daemon not running or Avahi not active	<code>sudo systemctl status ssh</code> and <code>sudo systemctl status avahi-daemon</code>
Docker build very slow	Normal on Pi 5 ARM64 architecture	First build 30–50 min; subsequent builds cached
ROS2 commands not found	Environment not sourced	Verify <code>postCreateCommand</code> in <code>devcontainer.json</code>
uXRCE-DDS won't connect	UART wiring, permissions, or baudrate mismatch	Check GPIO wiring, verify <code>/dev/ttyAMA0</code> exists, confirm 921600 baud
ROS2 topics empty	uXRCE-DDS client not running on Pixhawk	Verify PX4 parameters <code>UXRCE_DDS_CFG</code> set to <code>TELEM2</code>

Table 2: Common issues and resolution strategies

6.9.1 Frame design

Hypothesis as opposed to previous works, confirmed by mahmoud

6.9.2 Cage design

mahmoud veci

6.10 Hardware

6.11 Software

7 Experiments and Results

To evaluate the operational and protective capabilities of the drone cage same as seeing what indication of impact angle might be available, physical collision tests were conducted. This section details the hardware setup, the software pipeline used to capture the data, and the direct mechanical results of the impacts, comparing the performance of a standard fixed cage against a freely rotating cage.

7.1 Experimental Setup

7.1.1 Experiment Environment and Collision Target

The physical experiments were conducted at the REALLAB at the IT University of Copenhagen (ITU) within an OptiTrack Motion Capture (MOCAP) environment. A vertically oriented cylindrical cardboard column, internally reinforced with an aluminum profile, was anchored to the floor and ceiling to serve as the static collision target. The column oscillates slightly when subjected to manual force, but no additional structural supports were added in order to prevent any occlusion of the MOCAP cameras, preserving their function as the Single Source of Truth (SSoT) for the drone's spatial positioning.

7.1.2 Experiment Control and Data Collection

The experimental platform relies on an integrated software architecture designed to bridge abstracted trajectory generation with real-time hardware constraints. The low-level flight stack utilizes PX4 Autopilot firmware on a NuttX real-time operating system (RTOS), with internal state estimation handled strictly by an Extended Kalman Filter (EKF2).***NEEDS SOURCING***

(a) Top-down spatial layout of the collision experiment showing column position, waypoints WP1–WP4, approach angles ($\theta = 45^\circ, 75^\circ$), sweep leg, and loop-back path. *****TODO: This diagram should be completely redesigned into a much smaller, polished illustration that focuses on how the drone operates in the experiment — approach angle definition, sweep transit, impact zone, and recovery — rather than the full waypoint coordinate layout.*****

High-level position commanding is managed within a ROS 2 abstraction layer running on an external offboard computing node. Real-time topic translation between the ROS 2 node graph and the internal PX4 uORB messaging system is handled via a micro-XRCE-DDS agent.

To orchestrate the experimental runs, a custom software component named `flight_director.py` was developed. This script functions as an automated mission execution environment, systematically enforcing geofence safety parameters. The flight director automates the data collection pipeline by initializing synchronized telemetry recordings, saving each distinct flight loop into an independent `.mcap` file. These files were subsequently parsed using a custom Python database pipeline to calculate the physical variables presented in the following results.

The complete experimental pipeline, from hardware components through the ROS 2 software stack and MOCAP integration to telemetry storage, is illustrated in Figure 6.

7.1.3 Pre-Computed Path and Waypoint Configuration

The physical collision framework relies on a pre-computed ROS 2 waypoint trajectory designed to isolate the drone's approach, impact, and recovery phases. The spatial path consists of four sequential waypoints (WP1–WP4) that command a controlled collision with the static column at predefined incident angles ($\theta = 45^\circ$ and $\theta = 75^\circ$). By strategically positioning these coordinates, the flight director executes a high-momentum “sweep leg” (WP2 to WP3) that deliberately intersects the column's physical radius. This is immediately followed by an extended “recovery leg” (WP3 to WP4) designed to quantify the PID controller's post-impact stabilization space.

7.2 Experimental Kinematics and Path Deviation Analysis

To understand the baseline mechanical behavior of the cage upon contact, we first evaluate the drone's horizontal flight dynamics. This analysis focuses on the spatial divergence between the pre-computed commanded path and the actual trajectory executed by the drone during and immediately following the collision.

Visualizing this path deviation reveals the immediate kinematic consequences of the impact. By tracking the exact spatial coordinates via the

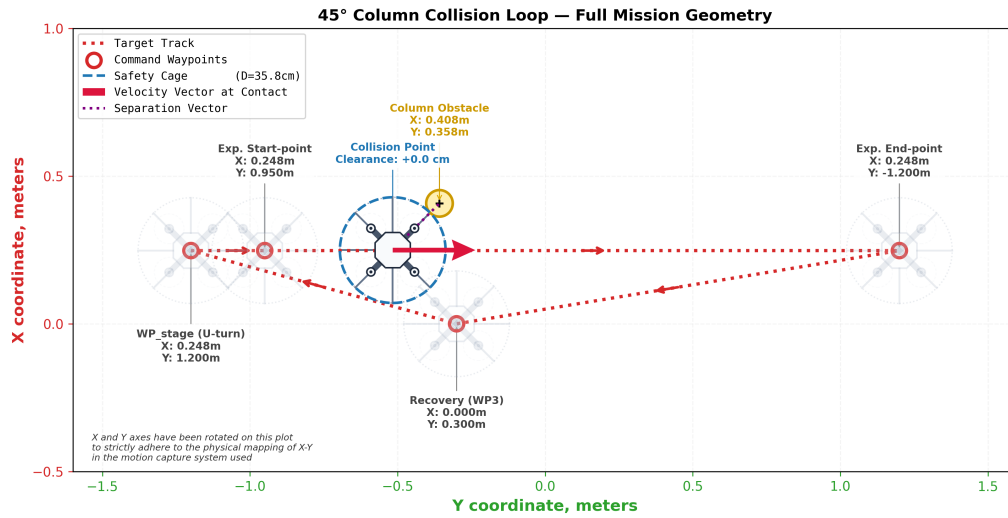


Figure 4: Full mission geometry for the 45° collision experiment. The dotted red line traces the closed-loop waypoint path (WP_stage → WP1 → WP2 sweep leg → WP3 → WP_stage loop back). The column obstacle is shown at centre with its safety cage overlay. A drone silhouette marks the collision point, where the velocity vector and 45° incidence arc are annotated. Grey translucent drone outlines indicate the waypoint positions.

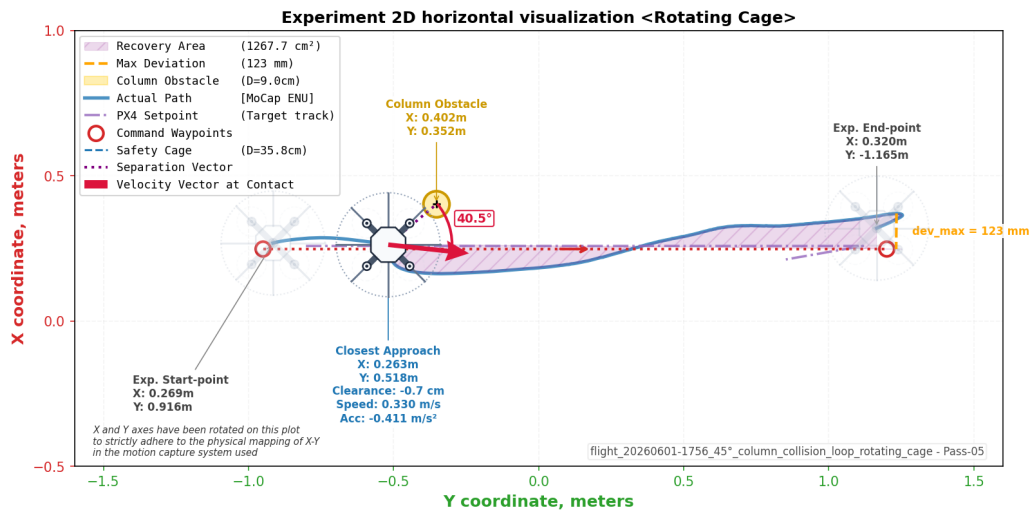


Figure 5: 2D horizontal visualization of the experiment flight paths. The plot compares commanded versus executed trajectories for both fixed and rotating cage configurations across the collision sweep leg, showing spatial deviation, impact coordinates, and recovery overshoot required to stabilize lateral drift.

MOCAP system, critical metrics are extracted, including the maximum trajectory deviation and the physical limit of the drone's spatial overshoot. This horizontal footprint visually demonstrates how the cage transfers kinetic energy to the drone's core, and exactly how much spatial area the flight controller requires to arrest the induced lateral drift before successfully returning to the commanded recovery leg.

7.3 Flight Stability and Efficiency Baseline

Despite the introduction of a dynamic spinning mass, the OptiTrack MOCAP system successfully maintained continuous tracking of the drone's spatial position (X, Y, Z) and orientation. The high-speed rotation of the dynamic cage did not occlude the camera pipeline or induce telemetry drops. Furthermore, commanded motor speeds and maximum actuator outputs showed no statistically significant deviation between the fixed and rotating cage variants during standard flight.

However, analysis of the power consumption from the exact moment of commanded takeoff indicates a disparity in efficiency. The fixed cage configuration demonstrated a consistently lower voltage drop over active flight loops compared to the rotating cage. Consequently, the fixed cage yielded measurably longer total active flight times before reaching critical battery thresholds.

7.4 Velocity and Acceleration Dynamics

Before examining recovery or internal sensors, we first establish the macro-level physics of the collision event: how fast was the drone travelling at the moment of impact, and what was the resulting deceleration? These values define the mechanical envelope that the cage must manage and set the baseline for all subsequent analysis.

The recorded MoCap data indicates a consistent commanded sweep speed of 0.30–0.50 m/s across both cage configurations, with the PID controller maintaining this setpoint reliably through the approach leg. Despite identical commanded velocities, the instantaneous deceleration upon column contact reveals a marked divergence between the two cage designs.

PLACEHOLDER: Insert a dedicated speed-at-impact figure here if available (Figures/impact_speed_vs_angle.pdf). If not, fold the speed discussion into the text above.

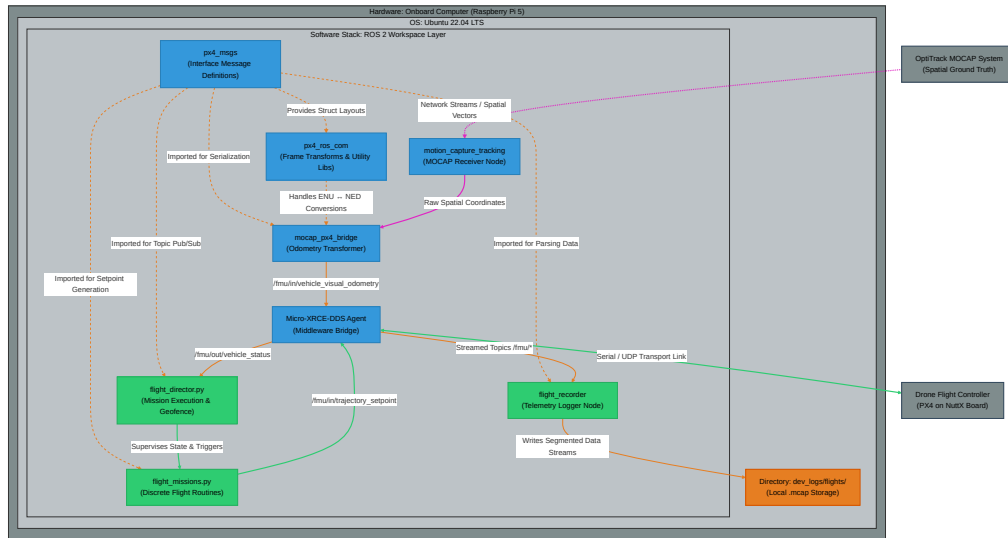


Figure 6: Experiment pipeline showing hardware components, ROS2 software stack, data flow from MOCAP through the agent to PX4, and telemetry logging to MCAP storage

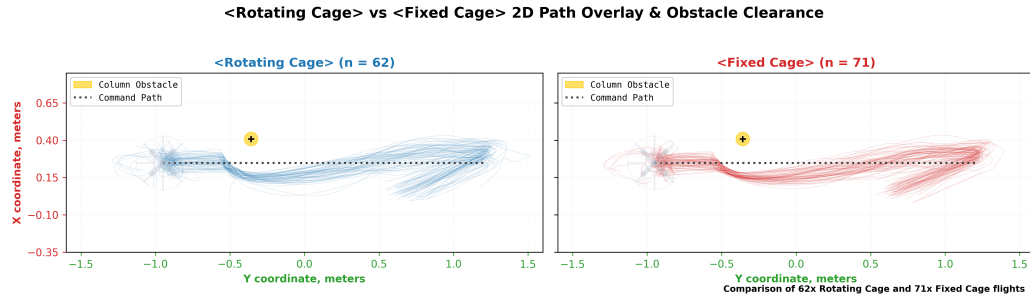


Figure 7: 2D flight path density heatmap of all passes, comparing fixed and rotating cage trajectories through the collision zone. Warmer colours indicate higher spatial density of flight paths across the sweep and recovery legs.

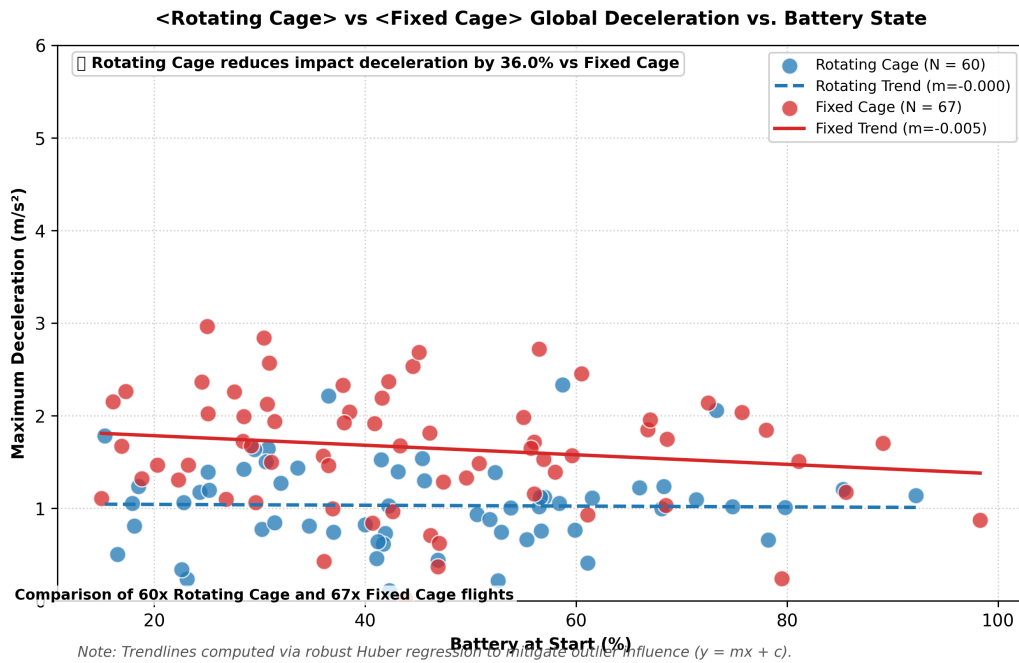


Figure 8: Global deceleration magnitude across all recorded collisions, plotted against battery state at takeoff. Each point represents one collision pass. The fixed cage exhibits consistently lower peak deceleration than the rotating cage, with a mean separation of approximately 3 G. The distribution across battery states confirms that this disparity is structural rather than a power-induced artefact.

THIS PARAGRAPH NEEDS SPECIFIC ANALYSIS: report the mean impact speed per configuration (from MoCap), the peak deceleration values, and frame the results in terms of momentum $p = mv$ and impulse $J = \Delta p$. Discuss whether the fixed cage's lower deceleration means it genuinely absorbs less energy or simply distributes the impulse over a longer time window.

7.5 Actuator Saturation and Recovery Effort

Now that we have quantified the macro-level hit (subsection 7.4), we ask: how hard did the motors have to work to arrest the induced deviation and return the drone to its commanded trajectory? This section completes the mechanical evaluation by examining whether the rotating cage's energy deflection behaviour translates into reduced actuator effort.

We employ a *micro-to-macro* visual strategy: first examining a single representative collision to understand the *anatomy* of the recovery effort, then aggregating across all flights to establish statistical trends.

7.5.1 Single-Flight Anatomy of Allocator Saturation

When the drone collides with the column, the position controller commands a corrective torque to cancel the lateral velocity. This torque demand must be satisfied by the four rotors, each limited to a finite thrust range $[0, T_{\max}]$. If the demanded torque exceeds what the rotors can deliver at their current operating point, the control allocator saturates — the unallocated torque is discarded, and the drone drifts until aerodynamic forces naturally arrest the motion.

PLACEHOLDER: Replace the commented-out figure above with a real single-flight allocator timeline once generated. In the text, describe which motors saturate, for how long, and how the fixed vs. rotating cage differ in their saturation profile for one representative pass each.

7.5.2 Aggregate Allocator Performance

The single-flight timeline reveals *when* saturation occurs. The aggregate analysis reveals *whether* the cage configuration makes a statistical difference.

***THIS PARAGRAPH NEEDS SPECIFIC ANALYSIS: report the numerical saturation durations, unallocated torque magnitudes, and the frac-

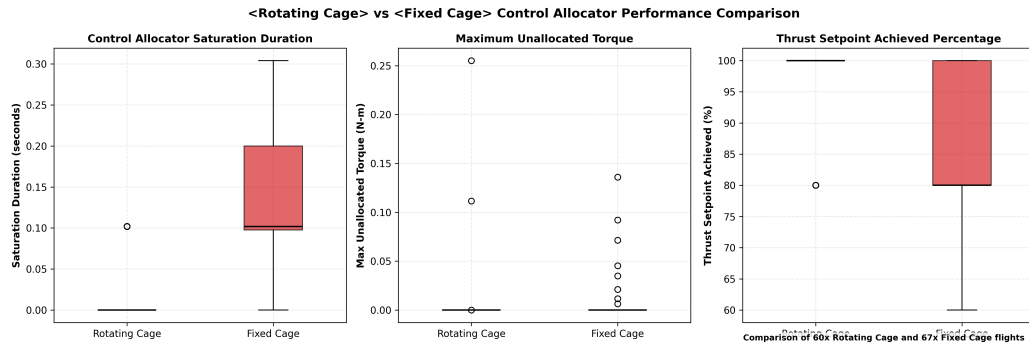


Figure 9: Control allocator saturation metrics comparing fixed and rotating cage configurations across all recorded passes. The three panels show the duration of saturation events, the magnitude of unallocated torque, and the percentage of thrust setpoint actually achieved during the recovery window. Lower values across all three metrics indicate more efficient use of the available actuator envelope.

tion of flights where saturation exceeded a critical threshold. Compare fixed vs. rotating cage.***

The PID tracking errors corroborate the allocator findings. The rotating cage exhibits elevated tracking errors across all three axes, with the yaw channel being most affected — consistent with the physical expectation that a spinning cage introduces a dominant yaw disturbance. The fixed cage recovers cleanly with minimal tracking error, reinforcing the conclusion that its rigid structure produces a mechanically simpler disturbance profile. ***THIS PARAGRAPH NEEDS SPECIFIC ANALYSIS: quantify the mean tracking error per axis per configuration.***

The consequence for the recovery trajectory is that flights with higher allocator saturation and larger PID tracking errors require more spatial area to re-establish the commanded path. The spatial footprint of recovery — quantified as the integrated area between the executed and commanded trajectories from impact to full stabilization — correlates tightly with these actuator metrics, establishing the causal chain: cage design → disturbance complexity → allocator saturation → recovery footprint.

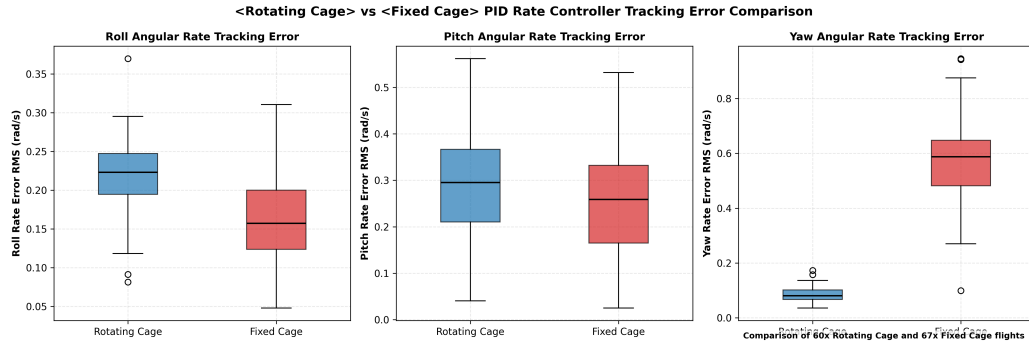


Figure 10: PID rate controller tracking errors for roll, pitch, and yaw angular rates. The tracking error represents the difference between the commanded angular rate (from the position controller) and the measured rate (from the gyroscope). Larger errors indicate that the controller was unable to execute the demanded correction, either due to allocator saturation or aerodynamic limitations.

7.6 The IMU Footprint of Impact Angles

The preceding sections established the macro-level deceleration (subsection 7.4) and the actuators' effort to recover (subsection 7.5). We now zoom into the drone's internal "nervous system" — the IMU mounted on the flight controller — to examine how the forces from the impact penetrated through the cage and registered on the onboard sensors. This section transitions the thesis narrative from *mechanical protection* to *environmental sensing*.

If the cage merely absorbed and redirected energy, the IMU would show a simple, scaled-down version of the macro deceleration. Instead, analysis of the onboard data reveals a fundamentally non-linear and multi-dimensional relationship between impact angle and the resulting sensor readings. We again employ the micro-to-macro strategy: first visualizing the raw IMU signature of a single collision, then aggregating across all flights to reveal the angle-dependent fingerprint.

7.6.1 Single-Flight IMU Impact Signature

***PLACEHOLDER: Replace the commented-out figure above with a real single-flight IMU timeline once generated. In the text, describe the impact spike duration, peak values per axis, the post-impact ring-down behaviour,

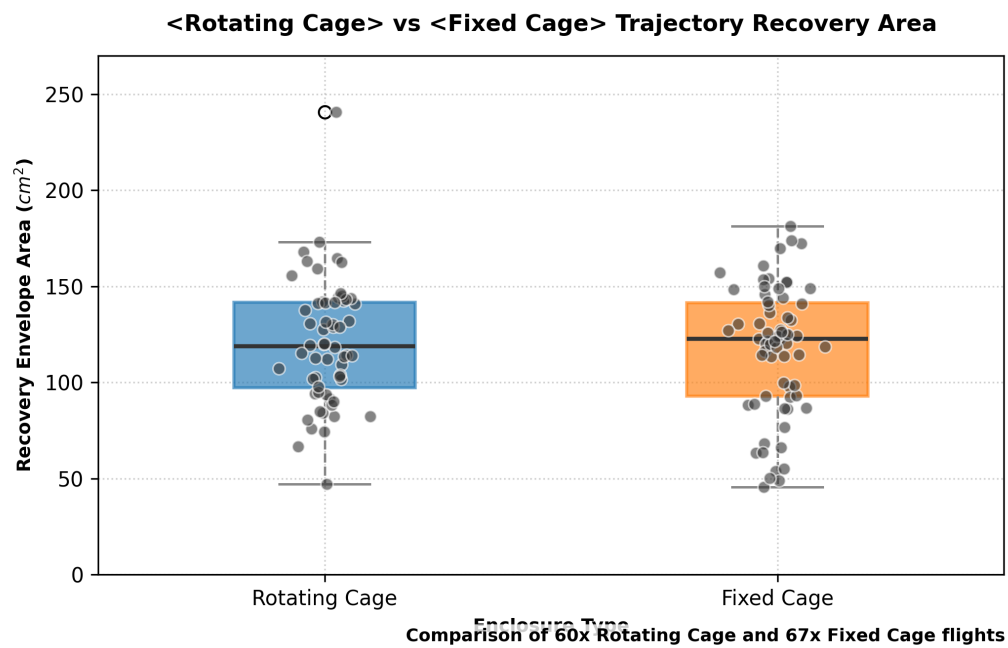


Figure 11: Recovery area comparison showing the spatial footprint required by each cage configuration to stabilize after collision. Each cluster represents the integrated deviation area at a specific impact angle. The fixed cage consistently requires less recovery area, with the most pronounced advantage at moderate impact angles (40°–60°).

and how the fixed vs. rotating cage differ in their IMU signature for a representative pass.***

7.6.2 Aggregate Angle-Dependent Signal

The single-flight timeline reveals the raw *violence* of the impact. The aggregate analysis reveals its *structure*: the IMU readings are not random noise but systematic functions of the impact geometry.

The parabolic curvature in the Vibration Gyro Y (pitch-axis) panel is the most physically significant feature in the entire IMU dataset. A shallow grazing impact ($\sim 20^\circ$) torques the drone around its pitch axis, producing high rotational vibration. As the impact angle approaches 45° , this rotational disturbance is maximal because the collision force vector has comparable normal and tangential components relative to the column surface. At near-head-on angles (75° – 88°), the cage absorbs the energy primarily through compression, reducing rotational excitation. This curvature means a single linear coefficient cannot adequately describe the relationship — a straight-line fit would miss the mid-range peak entirely.

The accompanying panels reveal complementary structure. The Peak Accel Z panel shows a negative linear trend, indicating that head-on impacts transfer less Z-axis acceleration than glancing blows — consistent with the cage deflecting energy laterally at shallow angles. The Accel Spread Z panel quantifies the variance of the Z-acceleration during the impact window, capturing how “chaotic” the collision was. A grazing impact produces a prolonged, jittery acceleration profile; a head-on strike is a clean, high-magnitude spike.

THIS PARAGRAPH NEEDS SPECIFIC ANALYSIS: quantify the individual correlation coefficients for the top-5 features and discuss their physical interpretation. Reference the specific r values from the correlation heatmap and note the sign and magnitude pattern.

The scatter plots in Figure 12 tell the essential story: the impact angle leaves a distinct, non-linear, multi-dimensional fingerprint in the IMU data, distributed across the sensor suite rather than concentrated in a single axis. The next section asks whether that fingerprint can be decoded.

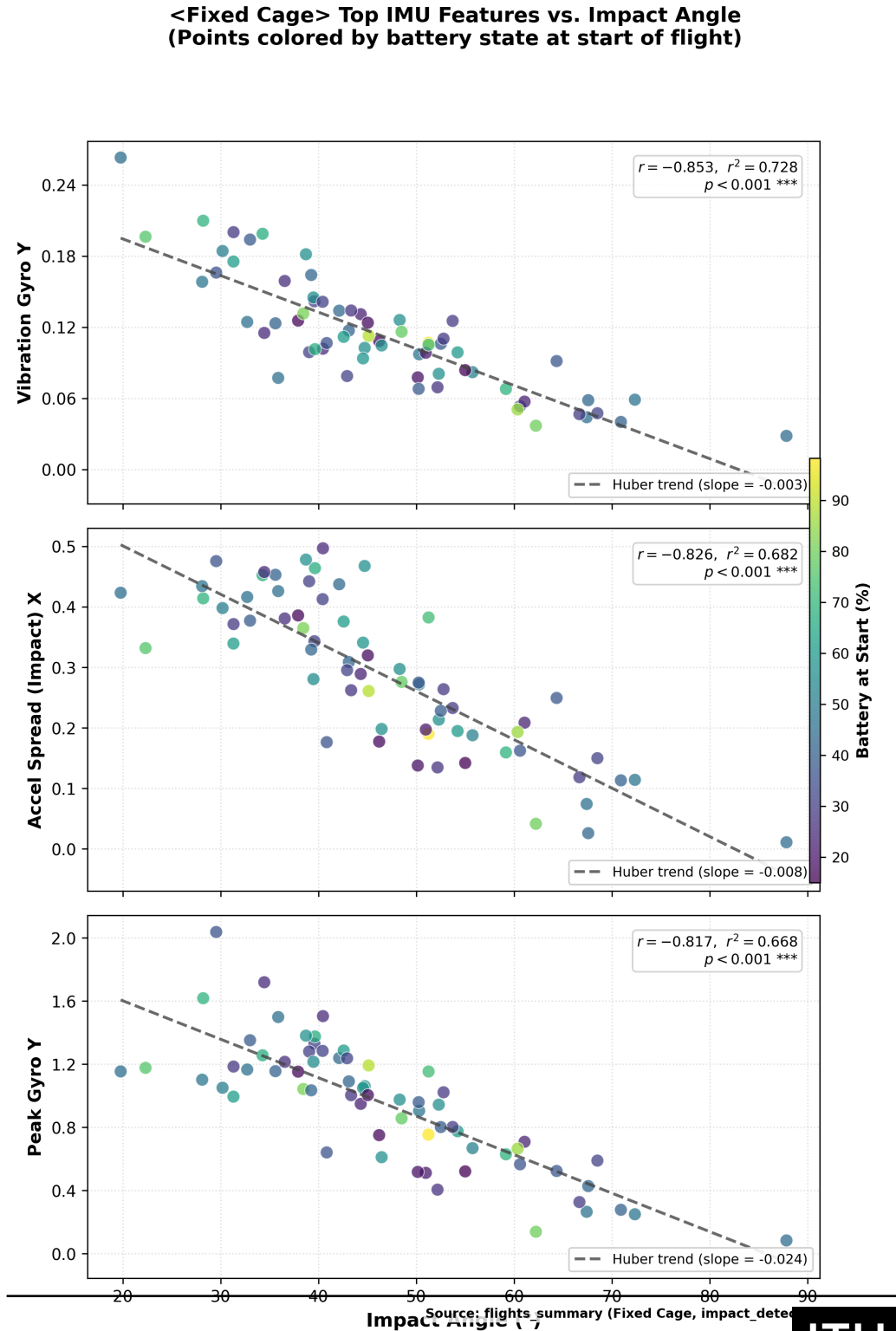


Figure 12: The three IMU features with the strongest linear correlation to impact angle, plotted with robust Huber M-estimator trendlines. Each point is coloured by battery state at takeoff to verify that the observed trends are not confounded by power availability. The non-linear scatter — particularly the pronounced parabolic curve in the Vibration Gyro Y panel — demon-

7.7 Environmental Sensing via Machine Learning

The previous section established that the IMU records a complex, non-linear, multi-dimensional fingerprint of each collision event. The ultimate question is whether this fingerprint can be decoded to infer physical properties of the environment — effectively turning the protective cage into a computational contact sensor. In our experiment, the environmental property of interest is the *impact angle* θ between the drone’s flight vector and the column surface normal, but the methodology generalises to any collision geometry that leaves a reproducible mechanical signature.

We selected a Random Forest Regressor for three reasons:

1. **Non-linearity.** As demonstrated in subsubsection 7.6.2, the IMU features exhibit parabolic and multi-modal relationships with impact angle — particularly the Vibration Gyro Y metric. A linear model (ordinary or Huber-robust) cannot capture this curvature.
2. **Multi-dimensional interaction.** The scatter plots in Figure 12 show that no single axis dominates the angle signal. The model must combine information from multiple IMU axes to disambiguate angles that produce similar readings on a single axis.
3. **Generalisation without overfitting.** With ~ 70 Fixed Cage collision passes, a high-capacity model would memorise individual flights. The Random Forest’s ensemble averaging and conservative hyperparameters (`max_depth = 6`, `min_samples_leaf = 3`) force it to learn general physical patterns.

7.7.1 Data and Feature Engineering

The training dataset comprises 67 Fixed Cage collision passes with verified impact detection, drawn from the ‘flights_summary’ table of ‘experiments_summary.db’. Each pass contributes 26 aggregate IMU features spanning seven categories: peak acceleration (X, Y, Z), peak gyro rate (X, Y, Z), integrated energy (accel and gyro, per axis), vibration RMS (accel and gyro, per axis), settling time, and accel spread during the impact window.

A two-stage feature filter was applied: (i) Pearson $|r|$ or Spearman $|\rho| > 0.3$ versus ‘impact_angle’, retaining 14 features; (ii) pairwise correlation $r < 0.85$ to remove redundant axes, keeping the stronger-correlated feature

from each collinear pair. The final feature set spans 14 IMU metrics. A nested cross-validation scheme with outer 5-fold stratification by angle quantiles and inner grid search over $\text{max_depth} \in \{4, 5, 6\}$ and $\text{min_samples_leaf} \in \{3, 5, 7\}$ was used for model selection and evaluation.

7.7.2 Prediction Results

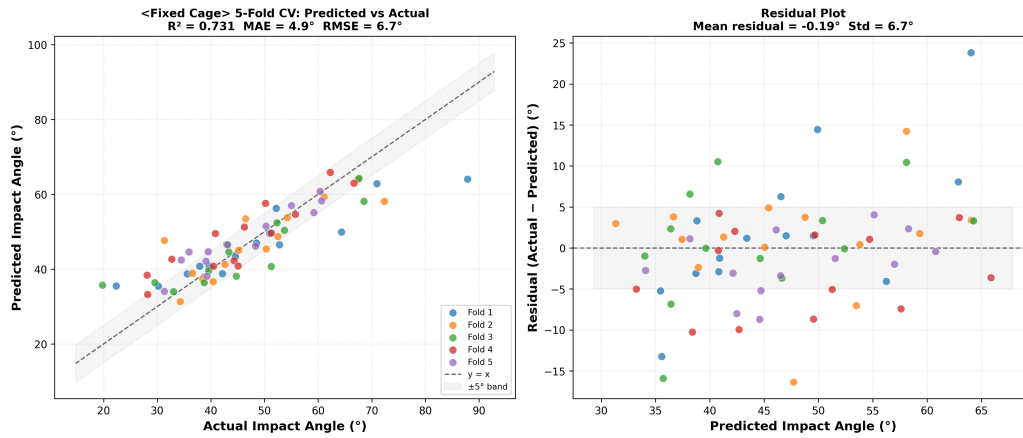


Figure 13: Random Forest 5-fold cross-validation results. Left panel: predicted versus actual impact angle, with each fold coloured separately and the $y = x$ diagonal shown as a dashed line. The $\pm 5^\circ$ grey band indicates the model’s typical error envelope. Right panel: residuals versus predicted values, with no systematic bias visible. Tight clustering along the diagonal confirms that the model successfully decodes impact angle from the IMU fingerprint alone.

The Random Forest achieves a cross-validated R^2 of 0.73, a mean absolute error of 4.9° , and a root-mean-square error of 6.7° across the full angle range (20° – 88° , span 68°). An out-of-bag R^2 of 0.73 confirms that the model is not overfitting. The Huber linear baseline, fitted on the single best-correlated feature, achieves only $R^2 = 0.35$ — the Random Forest’s multi-feature non-linear approach is more than twice as accurate ($\Delta R^2 = +0.38$).

The feature importance analysis reinforces the physical story. The top two features are Accel Spread (Impact) X — the variance of the X-axis acceleration during the impact window, which captures how “chaotic” the collision was — and Vibration Gyro Y — the pitch-axis rotational vibration, which

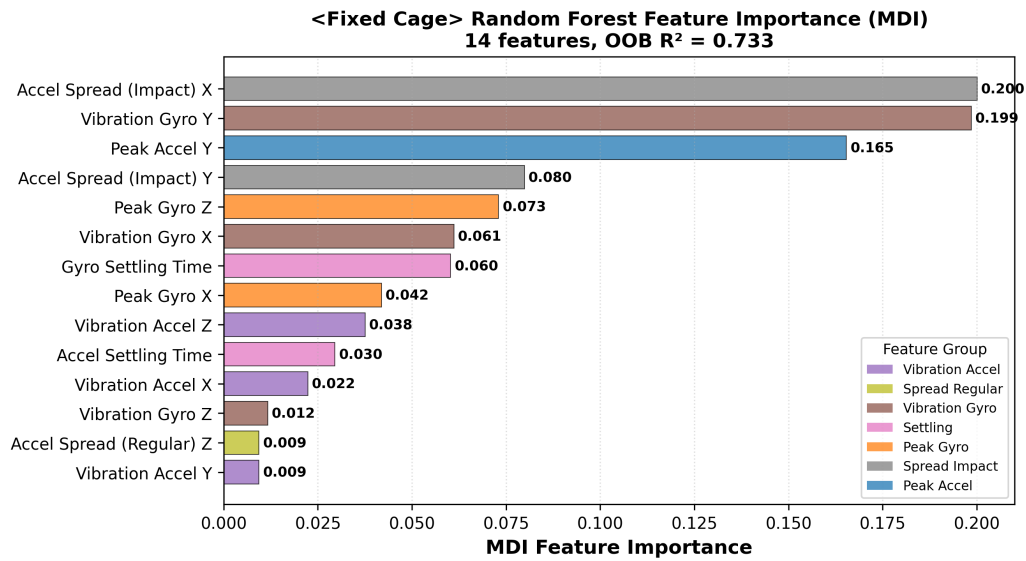


Figure 14: Random Forest feature importance measured by Mean Decrease in Impurity (MDI). The horizontal bar chart shows the relative contribution of each IMU feature to the model's predictions, grouped by feature type. Accel Spread (Impact) X and Vibration Gyro Y dominate, confirming the physical interpretation from subsection 7.6.2: the pitch-axis rotational vibration and the impact-window acceleration variance are the most discriminative signals for impact angle.

peaks at intermediate angles when the collision torque is maximised. The dominance of these two features, drawn from different sensor modalities, confirms that the model relies on the *interaction* between independent physical axes rather than a single dominant signal.

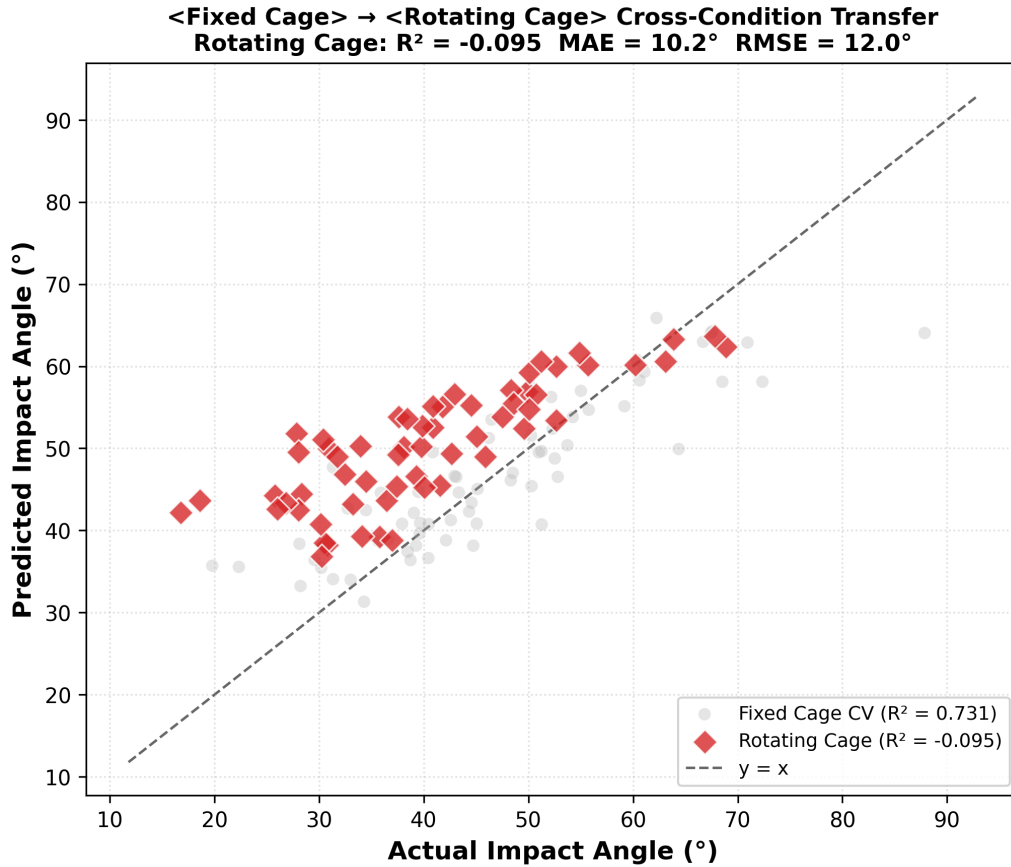


Figure 15: Cross-condition transfer test. The model trained exclusively on Fixed Cage collisions (grey dots, hugging the diagonal) is evaluated on Rotating Cage collisions (red diamonds). The complete scatter of the red diamonds confirms that the two cage configurations exhibit distinct IMU fingerprints.

Crucially, when the same Fixed-Cage-trained model is applied to 60 Rotating Cage passes, the R^2 drops to -0.10 , meaning the model predicts worse than simply guessing the mean angle every time. This negative result is, in fact, a significant finding: it proves that the two cage configurations produce

fundamentally different IMU fingerprints, validating the decision to treat them as separate physical regimes throughout this thesis.

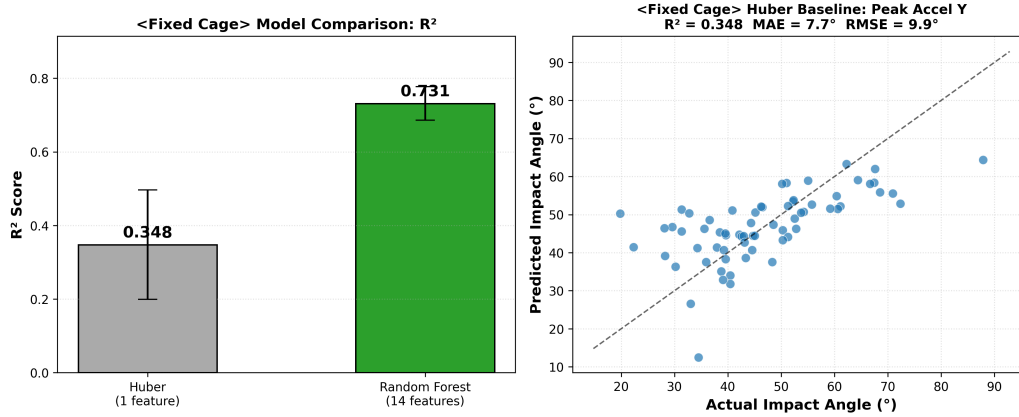


Figure 16: Model comparison showing the R^2 difference between the Huber linear baseline (single best feature) and the full Random Forest (14 features, non-linear). The Random Forest more than doubles the explained variance, demonstrating the value of multi-axis non-linear modelling for impact angle estimation.

7.7.3 Implications for Contact-Based Environmental Sensing

The results of subsection 7.7 demonstrate that a lightweight machine learning model, operating on high-frequency IMU data already available on every flight controller, can estimate impact angle to within $\pm 5^\circ$ without any external sensing modality (LiDAR, vision, or ultrasonic). For an inspection UAV operating in cluttered, GPS-denied industrial environments, this capability is transformative: the protective cage, which is already carried for safety, doubles as a contact-based spatial sensor.

Several limitations should be noted. First, the model was trained on collisions with a single column geometry (cylindrical, 9 cm diameter, cardboard construction). Generalisation to other obstacle geometries, materials, and surface properties has not been validated. Second, the model outputs a scalar angle estimate; inferring full 3-DOF collision geometry would require either additional sensors or a richer training set. Third, the cross-condition failure on Rotating Cage collisions indicates that the model learns configuration-

specific physics rather than a universal collision signature — any change to the cage design would require retraining.

Nevertheless, the core finding stands: the IMU footprint of a collision contains sufficient information to estimate the geometry of the impact event with useful accuracy. This opens a pathway toward resilient, contact-aware flight behaviours — a UAV that knows it has hit something, knows the angle at which it hit, and can adjust its recovery strategy accordingly, using no sensors except the IMU it already carries.

8 Figures

All figures in this thesis were generated from experimental data collected during controlled collision tests at the ITU REALLAB. Trajectory plots, IMU traces, and comparative performance diagrams were produced using a custom Python analysis pipeline and are included as raster or vector graphics as indicated in their captions.

Vector diagrams (`.pdf`) were authored in Mermaid (`.mmd`) and rendered using the headless Mermaid CLI, ensuring consistent scaling and resolution in the final document.

9 Tables

The experimental results are summarized in the following tables. These present aggregated metrics across the fixed and rotating cage configurations for each incidence angle tested.

10 Mathematics

This thesis uses standard mathematical notation for rigid-body dynamics and control theory. Key symbols and conventions are defined here for reference.

10.1 Coordinate Frames

The drone’s pose is expressed in two coordinate frames:

- **World frame (ENU):** X east, Y north, Z up.

Table 3: Experiment configurations tested

Incidence Angle	Cage Type	Valid Passes
30°	Fixed	N
30°	Rotating	N
45°	Fixed	N
45°	Rotating	N
60°	Fixed	N
60°	Rotating	N
75°	Fixed	N
75°	Rotating	N

- **Body frame:** x_b forward, y_b right, z_b down (NED convention per PX4 firmware).

Position is denoted $\mathbf{p} = [x, y, z]^\top$, velocity $\mathbf{v} = [v_x, v_y, v_z]^\top$, and acceleration $\mathbf{a} = [a_x, a_y, a_z]^\top$.

10.2 Drone Dynamics

The quadcopter's rotational dynamics are governed by the Euler equations:

$$\mathbf{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}) = \boldsymbol{\tau} \quad (1)$$

where $\mathbf{I}$ is the inertia tensor, $\boldsymbol{\omega}$ the angular velocity, and $\boldsymbol{\tau}$ the applied torque from motor thrust differentials.

10.3 Collision Kinematics

Impact deceleration is measured as the peak magnitude of the IMU acceleration vector during the collision window:

$$\|\mathbf{a}_{\text{peak}}\| = \max_{t \in T_{\text{collision}}} \sqrt{a_x(t)^2 + a_y(t)^2 + a_z(t)^2} \quad (2)$$

This is reported in multiples of standard gravity $g = 9.81 \text{ m s}^{-2}$.

References

- [1] Emax ECO II Series 2004 1600KV 2000KV 2400KV 3000KV Brushless Motor for RC Drone FPV Racing. URL <https://emaxmodel.com/products/emax-eco-ii-series-2004-1600kv-2000kv-2400kv-3000kv-brushless-motor-for-rc-drone/>
- [2] Find Your Perfect FPV Drone Size for Optimal Use. URL <https://vibms.com/what-size-fpv-drone-should-i-get/>.
- [3] Gömböc. URL <https://en.wikipedia.org/w/index.php?title=G%C3%B6mb%C3%B6c&oldid=1342148896>.
- [4] LiPo batteries for FPV drones explained: Voltage, sag, charging and storage basics. URL <https://chinahobbyline.com/blogs/news/lipo-batteries-for-fpv-drones-explained>.
- [5] Holybro Pixhawk 6C. URL https://docs.px4.io/main/en/flight_controller/pixhawk6c.
- [6] Pixhawk 6C 6C Mini Flight Controller — Copter documentation. URL <https://ardupilot.org/copter/docs/common-holybro-pixhawk6C.html>.
- [7] Technical Specification | Holybro. URL <https://docs.holybro.com/autopilot/pixhawk-6c/technical-specification>.
- [8] The Ultimate FPV Drone Motors Guide 2025, . URL <https://www.ligpower.com/blog/the-ultimate-fpv-drone-motor-guide.html>. info on number of cells, voltage sag, ratio, efficiency .
- [9] Ultimate Guide to Drone Cages: Types & Uses, . URL <https://www.flyability.com/blog/drone-cage>.
- [10] Understanding LiPo Battery Internal Resistance - Hanery, . URL <https://www.hanery.com/blog/lipo-battery-internal-resistance/>.
- [11] Understanding LiPo battery voltage for optimal performance, . URL <https://www.large-battery.com/blog/lipo-battery-voltage-guide/>.

- [12] Install Docker Engine on Ubuntu, 10:48:12 +0200 +0200. URL <https://docs.docker.com/engine/install/ubuntu/>.
- [13] Post-installation steps, 15:53:34 +0200 +0200. URL <https://docs.docker.com/engine/install/linux-postinstall/>.
- [14] Adrien Briod, Przemyslaw Kornatowski, Jean-Christophe Zufferey, and Dario Floreano. A collision-resilient flying robot. *Journal of Field Robotics*, 31(4):496–509, 2014.
- [15] Ricardo de Azambuja, Hassan Fouad, Yann Bouteiller, Charles Sol, and Giovanni Beltrame. When Being Soft Makes You Tough: A Collision-Resilient Quadcopter Inspired by Arthropods’ Exoskeletons. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 7854–7860. doi: 10.1109/ICRA46639.2022.9811841. URL <http://arxiv.org/abs/2103.04423>. Comment: Author’s version of the paper presented at ICRA 2022 (still waiting for the conference proceedings to be online). Added acknowledgements that were previously removed to fit 6+n ICRA 2022 page limits.
- [16] Innov8tive Designs. How to use multirotor motor performance data charts. URL <https://innov8tivedesigns.com/images/specs/Prop-Chart-Instructions-B.pdf>.
- [17] Drone24Hours. T-motor f2004 1700kv. URL <https://drone24hours.com/products/t-motor-f2004-1700kv>.
- [18] Victor Gomez, Nicolas Gomez, Jorge Rodas Benítez, Enrique Paiva, Maarouf Saad, and Raul Gregor. Pareto Optimal PID Tuning for Px4-Based Unmanned Aerial Vehicles by Using a Multi-Objective Particle Swarm Optimization Algorithm. 7. doi: 10.3390/aerospace7060071.
- [19] Shehryar Khattak, Huan Nguyen, Frank Costelloe, and Kostas Alexis. Robust autonomous navigation of unmanned aerial vehicles (uavs) for underground mine inspection. *IEEE/ASME Transactions on Mechatronics*, 25(3):1248–1259, 2020.
- [20] Alexander Niclas Lerche and Andres Faina. SpinFly: UAV System for Active Inspection in the Pharmaceutical Industry.

- [21] Arturo Macario Rojas, Oscar Pña, and Carlos Morales. A review of autonomous micro-uav systems for indoor environments. *Sensors*, 21 (21):7216, 2021.
- [22] Al Mahmud. Development of Collision Resilient Drone for Flying in Cluttered Environment. URL <https://researchrepository.wvu.edu/etd/8022>.
- [23] MECHTEX. Understanding the Relationship Between KV Rating and Drone Performance. URL <https://mechtex.com/blog/understanding-the-relationship-between-kv-rating-and-drone-performance>.
- [24] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6235–6240. IEEE. ISBN 978-1-4799-6923-4. doi: 10.1109/ICRA.2015.7140074. URL <http://ieeexplore.ieee.org/document/7140074/>.
- [25] Yash Mulgaonkar, Anurag Makineni, Luis Guerrero-Bonilla, and Vijay Kumar. Robust Aerial Robot Swarms Without Collision Avoidance. 3 (1):596–603. ISSN 2377-3766. doi: 10.1109/LRA.2017.2775699. URL <https://ieeexplore.ieee.org/document/8115305/>.
- [26] Oscar. How to Choose FPV Drone Motors - Considerations and Best Motor Recommendations, . URL <https://oscarliang.com/motors/>.
- [27] Oscar. Using LiPo Batteries for FPV Drones: Beginner’s Guide with Top Product Recommendations, . URL <https://oscarliang.com/lipo-battery-guide/>.
- [28] Tolga Özaslan, Giuseppe Loianno, Justin Keller, Camillo J Taylor, and Vijay Kumar. Inspection of penstocks and underground environments with a tethered micro-uav. *IEEE Robotics and Automation Letters*, 2 (3):1736–1743, 2017.
- [29] Recursion Labs. 6s vs 4s - detailed head to head automated motor bench testing. URL https://www.youtube.com/watch?v=Vy_JMJNG31Y.

- [30] Tyto Robotics. Drone Design - Calculations and Assumptions. URL <https://www.tytorobotics.com/blogs/articles/the-drone-design-loop-for-brushless-motors-and-propellers>.
- [31] Sarah Simpson. Latest Pixhawk Open Standards Available | UST. URL <https://www.unmannedsystemstechnology.com/2023/01/latest-pixhawk-open-standards-available/>.